

Application-Managed Flash

FTL Study

2026. 08. 11

Presentation by 정다원, 김정민

clio6613@dankook.ac.kr

Contents

1. Introduction
2. Abstract
3. ALFS
4. AFTL
5. Garbage Collection
6. Benchmark
7. Related Work
8. Conclusion

1.1. FTL이 하는일



FTL은 In-Place Update를 피하고,
NAND의 낮은 신뢰성을 관리하기 위해
복잡한 작업을 수행한다.

1.1. FTL이 하는 일



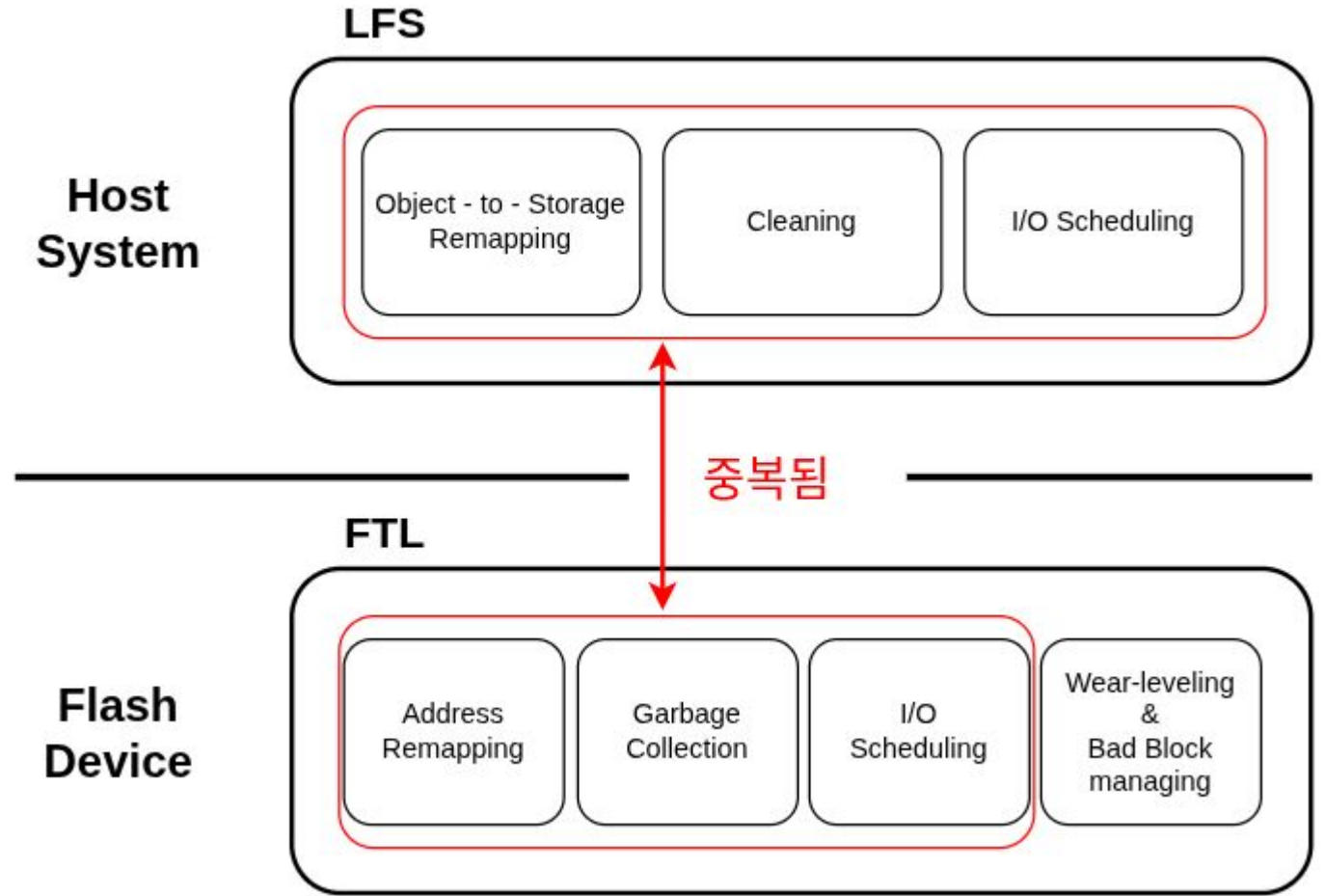
해당 작업으로 인한 Trade-off

- 상당한 양의 H/W 자원 사용
- 추가적인 I/O 발생
- Disk의 동작을 예측하기 어려워짐

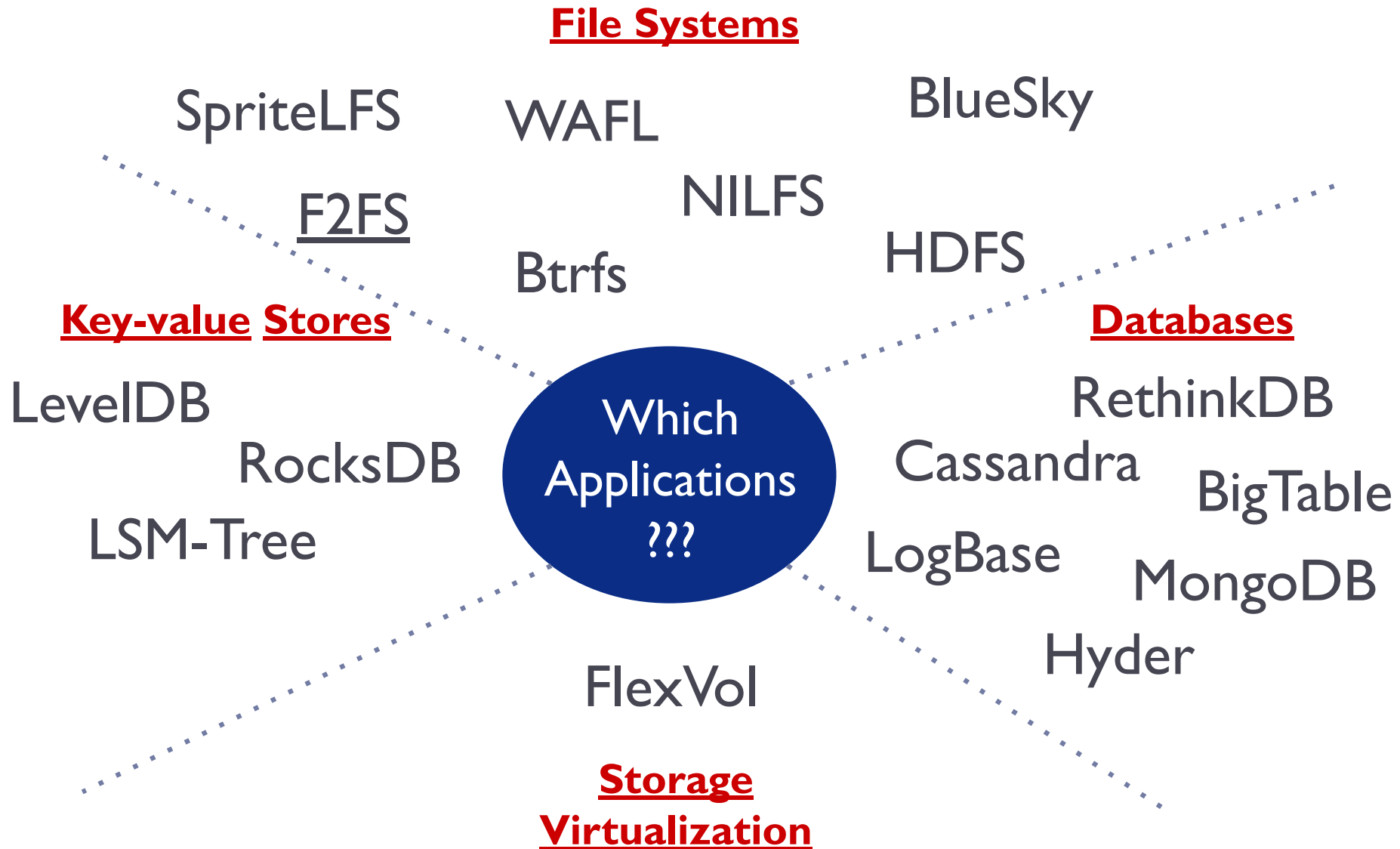
1.2. Semantic Gap

하지만 이미 존재하는 Application들과 겹치는 기능들이 다수.

FTL과 Host는 서로의 상태를 모르기 때문에, 겹치는 기능들이 서로 제각각 동작하게 된다.



1.3. NAND Friendly Applications



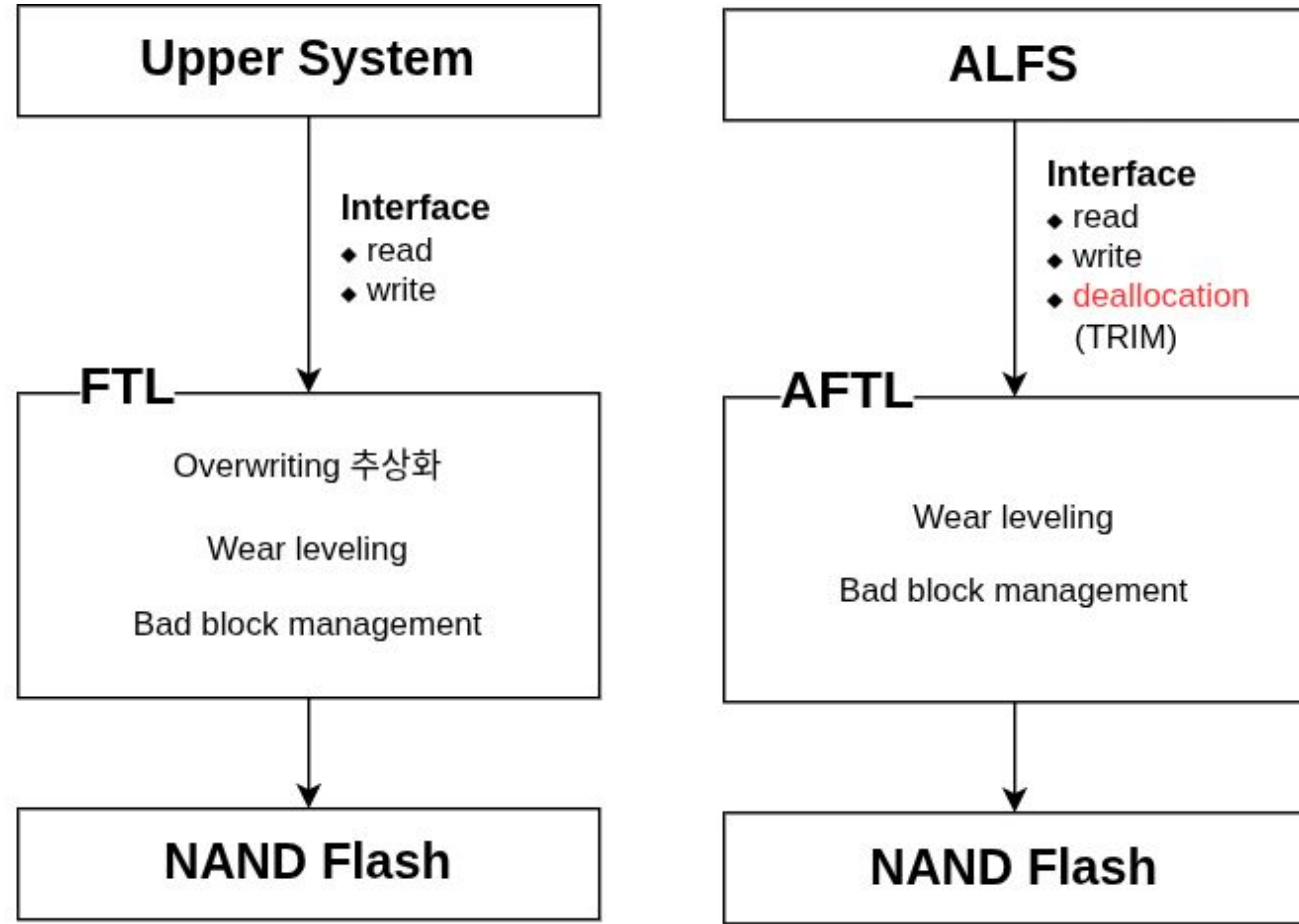
2.1. AMF의 해결책

**Host system이 flash 친화적이라면,
FTL의 복잡한 작업을 Host system에게 맡긴다.**

2.2. AMF Overview

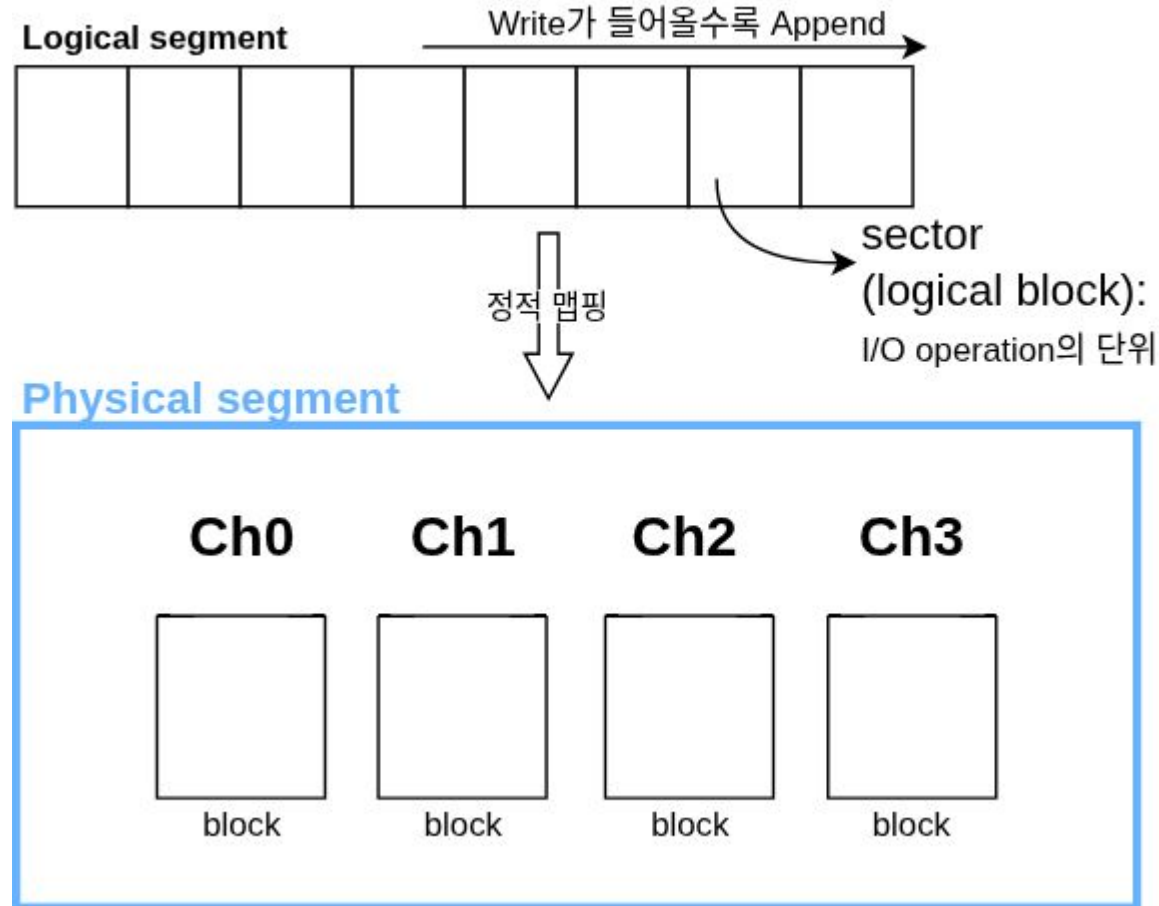
TRIM에 관련한 2가지 원칙

- 1. TRIM 되기 전까진 Sector를 다시 쓸 수 없다.
- 2. TRIM은 반드시 Segment 단위로 수행.



기존 방식(좌측)과 AMF의 방식(우측)

2.3. AMF Structure

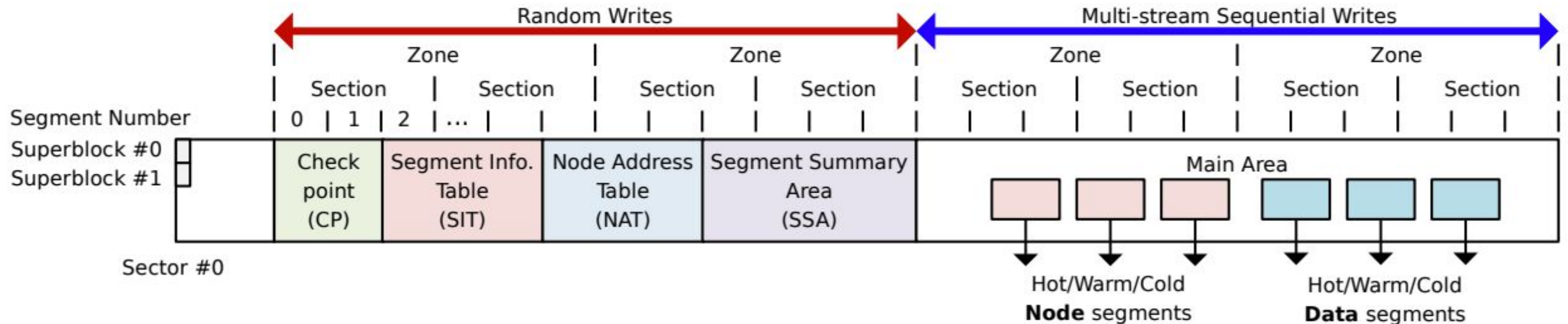


정적 맵핑:

한번 할당된 물리적 위치는
변하지 않음

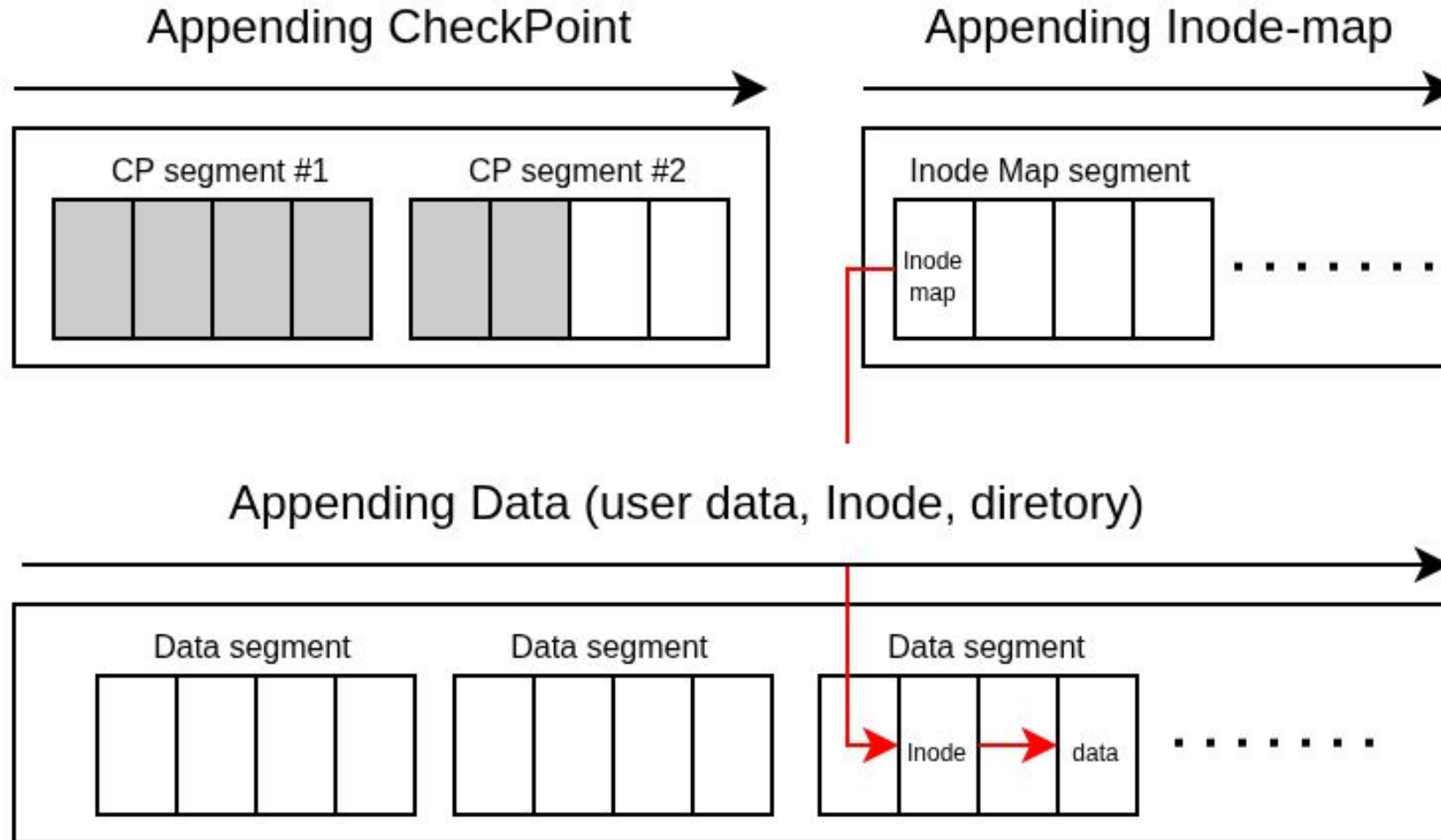
3.1. F2FS (Flash-Friendly File System)

ALFS는 F2FS의 구조를 기반으로 한다.



F2FS는 기존 LFS에 비해 여러 개선 사항이 존재하지만,
핵심 구조는 데이터의 특성에 따라 서로 다른 Segment로 분리하고,
Multi-head Logging을 통해 각 Segment를 독립적으로 기록.

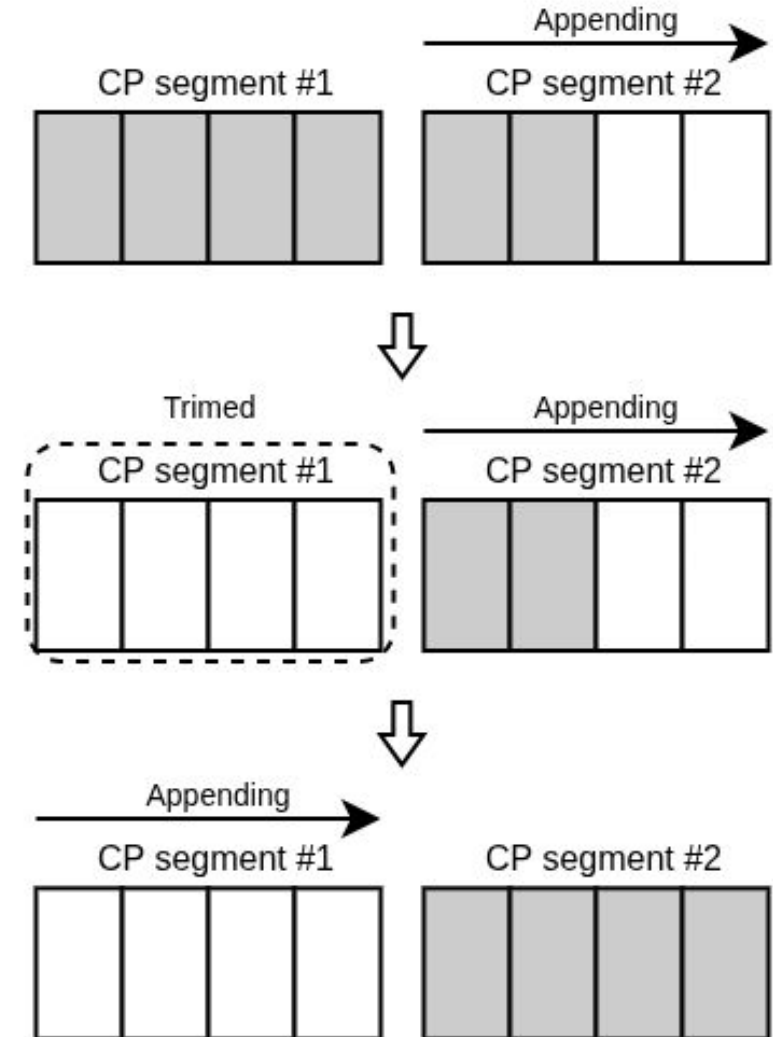
3.2. ALFS Structure



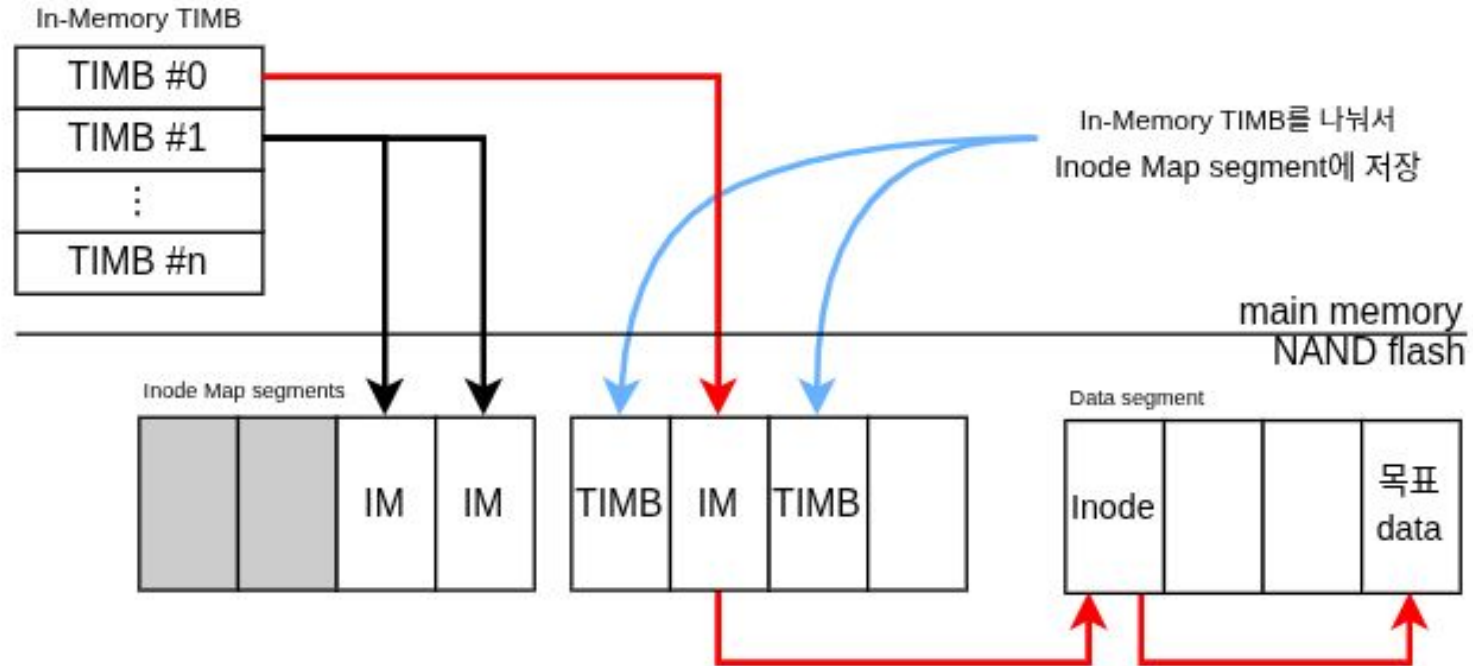
3.3. CheckPoint Appending

두개의 segment를 번갈아가면서
한쪽은 TRIM, 한쪽은 Appending 된다.

Recovery시,
2개의 segment만 탐색하기에
탐색 시간을 줄일수 있게된다.



3.4. TIMB (Table for Inode Map Blocks)



Inode Map의 빠른 탐색을 위해 TIMB를 메모리에 유지한다.

4.1. FTL Mapping

segment당
4개의 block을 포함

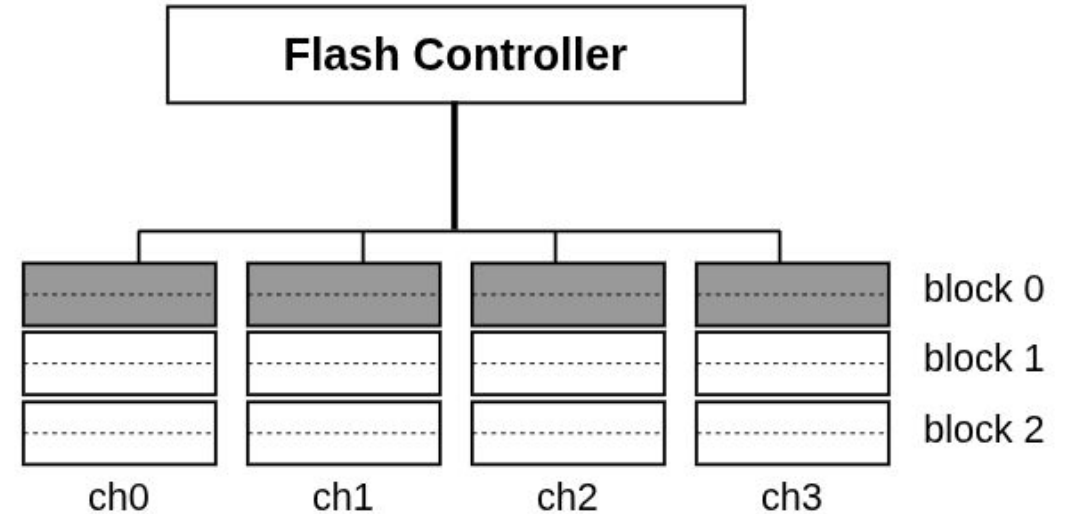
block당 2개의
page를 포함

sector와 Page는
1대1 대응

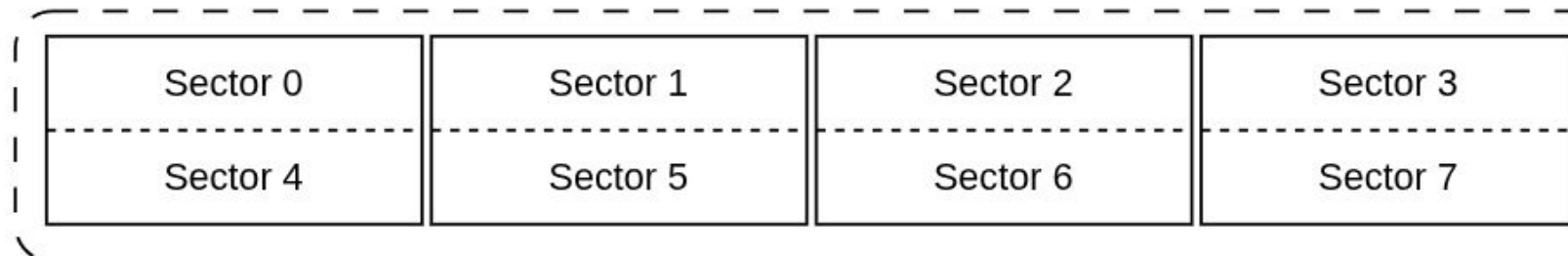
→ 1개의 Segment에
8개의 섹터 포함

Segment-Map Table
(STAT | channel, way, block)

segment #99 (TRIM됨)	Invalid	(0, 0, 0)
	Invalid	(1, 0, 0)
	Invalid	(2, 0, 0)
	Invalid	(3, 0, 0)
segment #100	Used	(0, 0, 1)
	Used	(1, 0, 1)
	Used	(2, 0, 2)
	Used	(3, 0, 1)
segment #101 (할당 안됨)	Free	(-1, -1, -1)
	Free	(-1, -1, -1)
	Free	(-1, -1, -1)
	Free	(-1, -1, -1)



Sector to Physical mapping detail



4.1. FTL Mapping

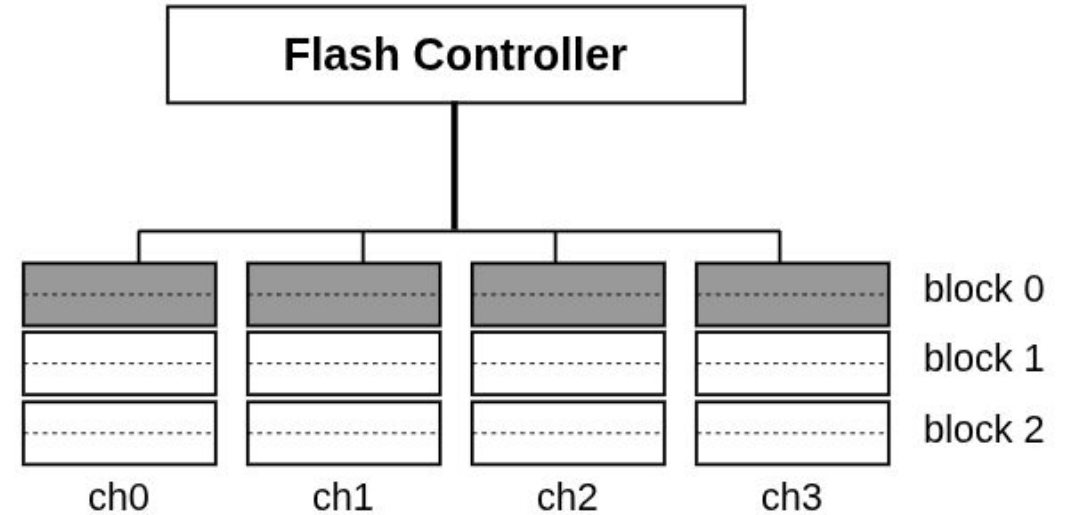
ex) sector #801

Segment:
 $801/8 = 100$

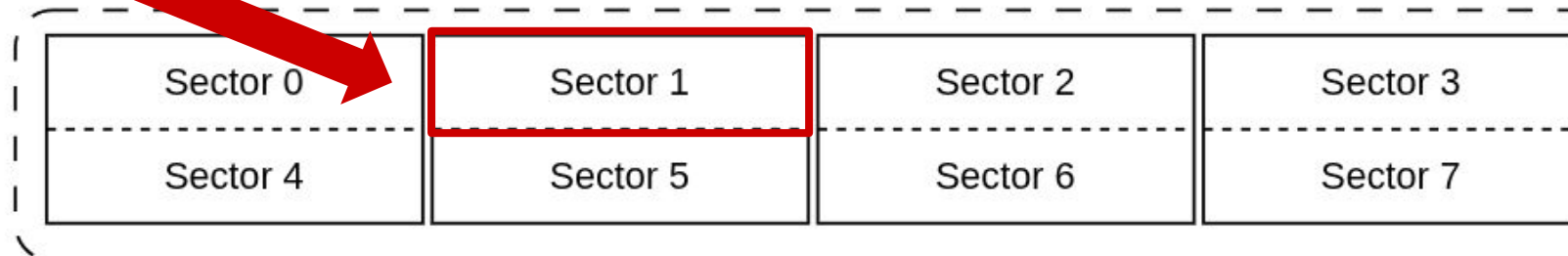
Offsett:
 $801 \% 8 = 1$

Segment-Map Table
(STAT | channel, way, block)

segment #99 (TRIM됨)	Invalid	(0, 0, 0)
	Invalid	(1, 0, 0)
	Invalid	(2, 0, 0)
	Invalid	(3, 0, 0)
segment #100	Used	(0, 0, 1)
	Used	(1, 0, 1)
	Used	(2, 0, 2)
	Used	(3, 0, 1)
segment #101 (할당 안됨)	Free	(-1, -1, -1)
	Free	(-1, -1, -1)
	Free	(-1, -1, -1)
	Free	(-1, -1, -1)



Physical mapping detail

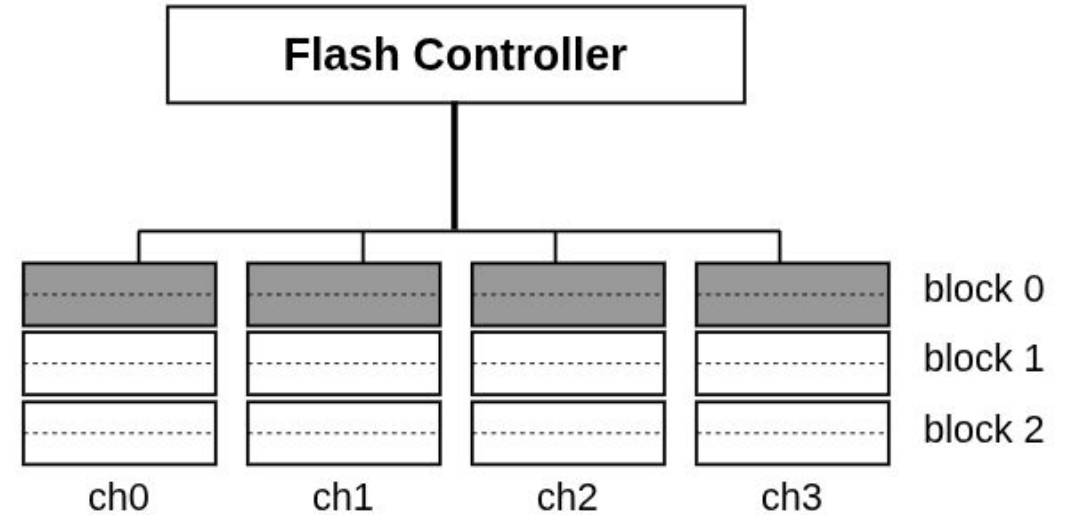


4.1. FTL Mapping

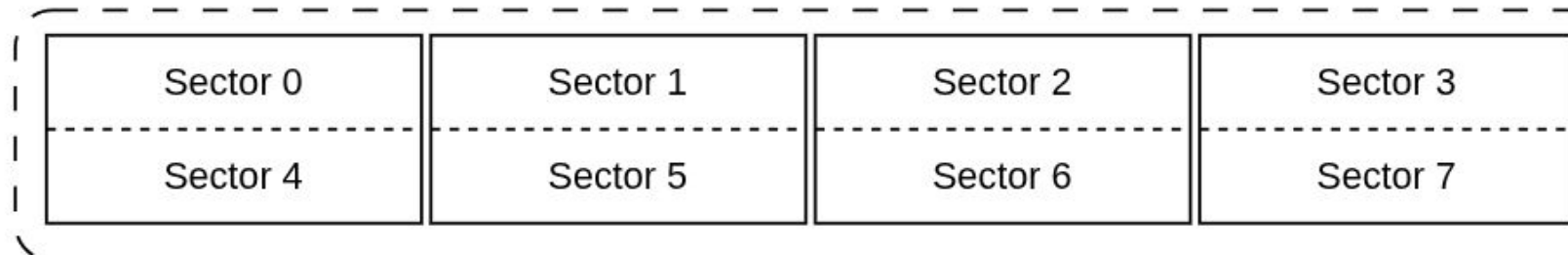
ex) sector #801

Segment-Map Table		
(STAT channel, way, block)		
segment #99 (TRIM됨)	Invalid	(0, 0, 0)
	Invalid	(1, 0, 0)
	Invalid	(2, 0, 0)
	Invalid	(3, 0, 0)
segment #100	Used	(1, 0, 1)
	Used	(2, 0, 2)
	Used	(3, 0, 1)
segment #101 (할당 안됨)	Free	(-1, -1, -1)
	Free	(-1, -1, -1)
	Free	(-1, -1, -1)
	Free	(-1, -1, -1)

해당 블록의 첫번째 페이지



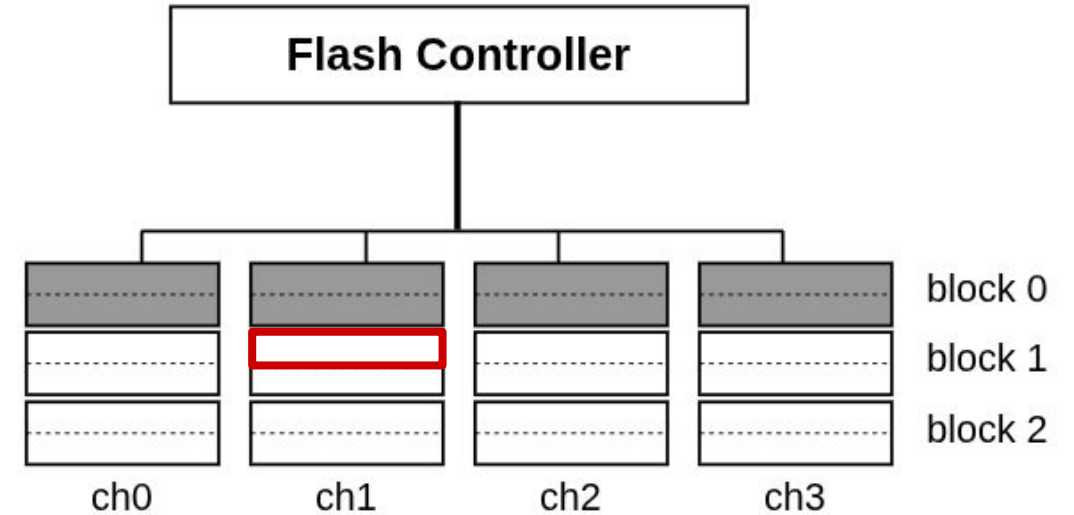
Sector to Physical mapping detail



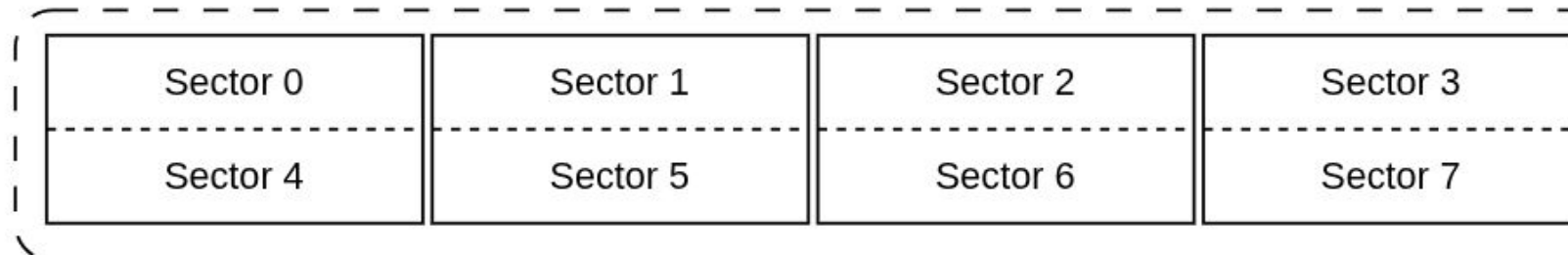
4.1. FTL Mapping

ex) sector #801

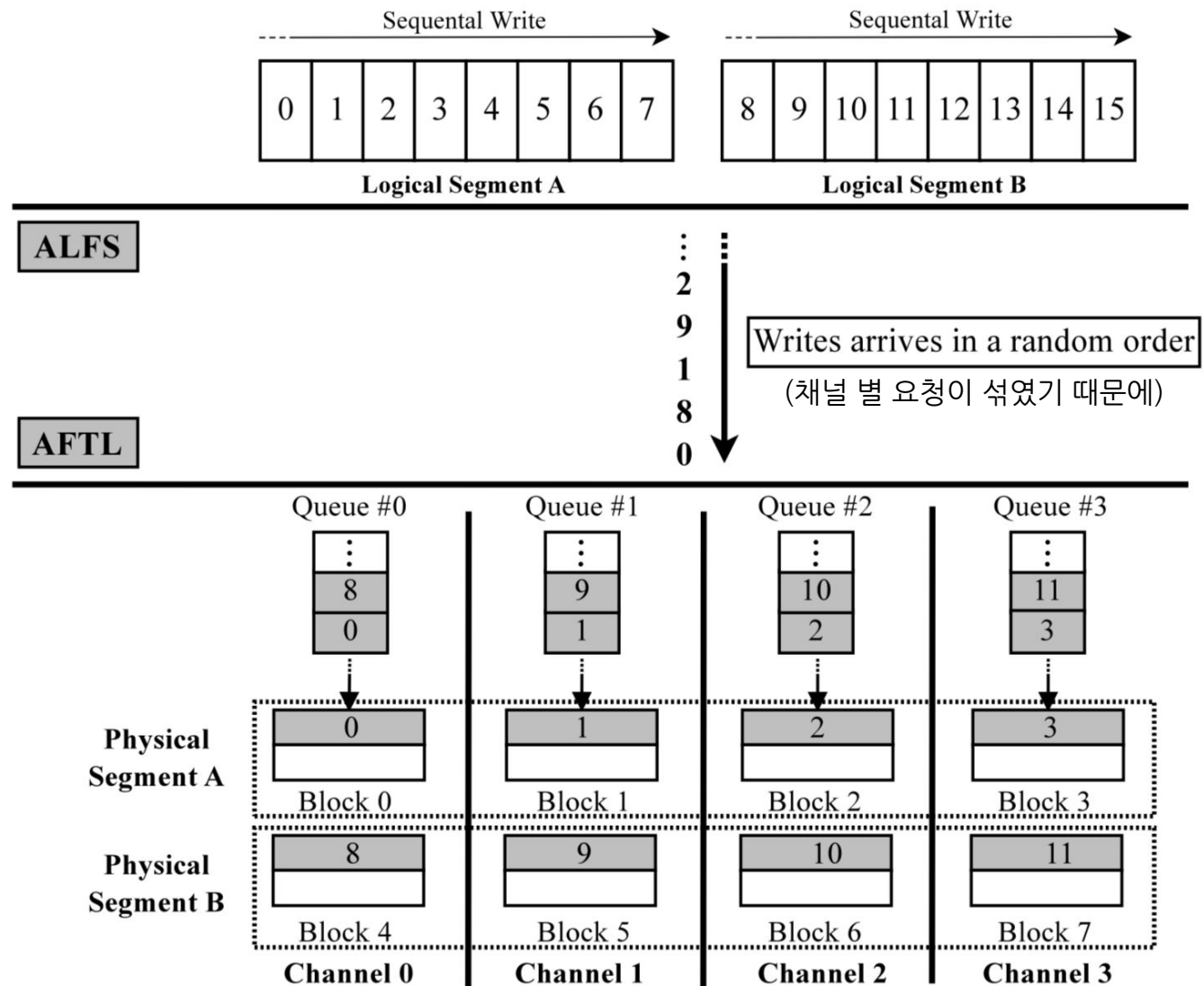
Segment-Map Table		
(STAT channel, way, block)		
segment #99 (TRIM됨)	Invalid	(0, 0, 0)
	Invalid	(1, 0, 0)
	Invalid	(2, 0, 0)
	Invalid	(3, 0, 0)
segment #100	Used	(0, 0, 1)
	Used	(1, 0, 1)
	Used	(2, 0, 2)
	Used	(3, 0, 1)
segment #101 (할당 안됨)	Free	(-1, -1, -1)
	Free	(-1, -1, -1)
	Free	(-1, -1, -1)
	Free	(-1, -1, -1)



Sector to Physical mapping detail



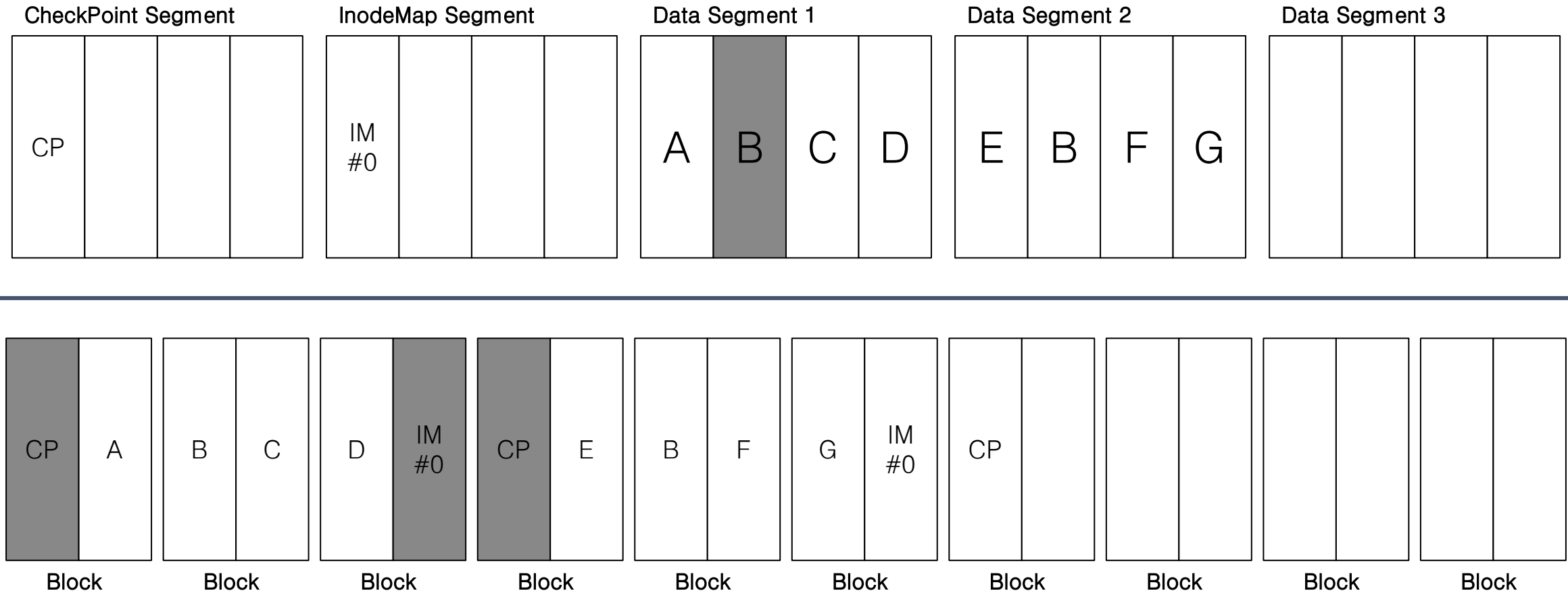
4.2. 채널별 QUEUE



채널별 큐를 통해 요청 재정렬

5. Garbage Collection (F2FS + FTL)

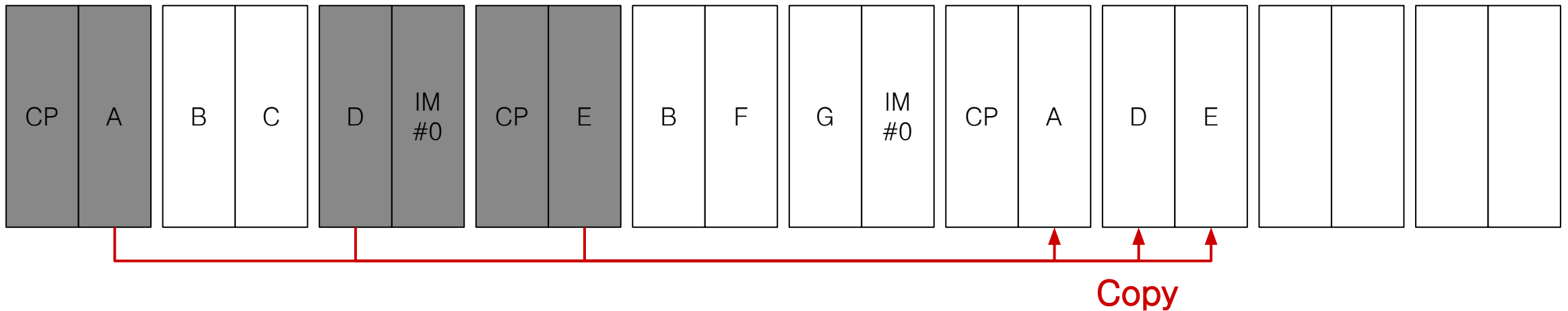
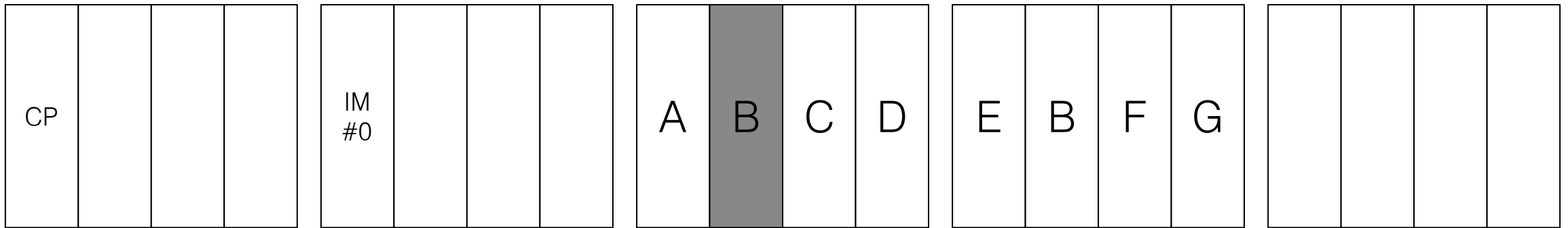
0. 초기 상태



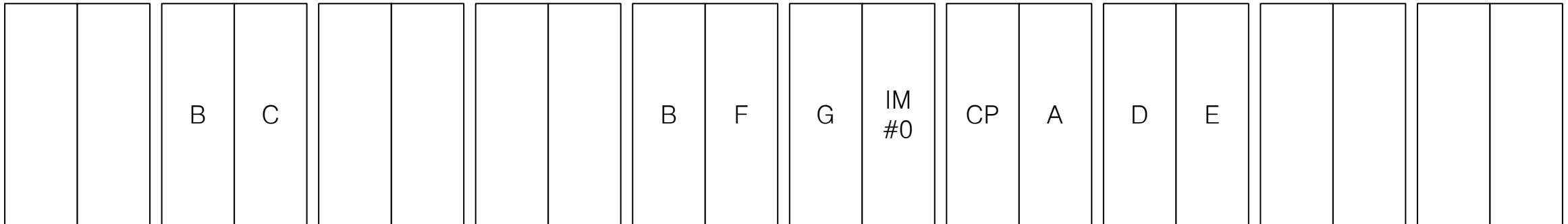
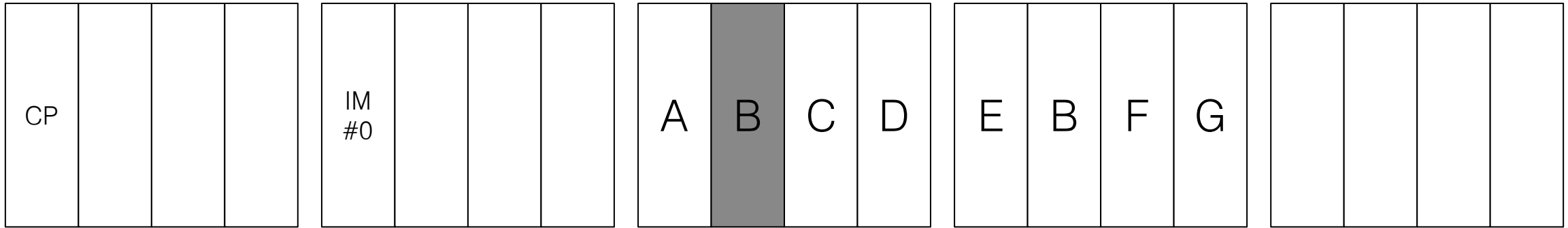
(FTL의 GC trigger: Clean block이 3개 이하일 때)

5. Garbage Collection (F2FS + FTL)

1. FTL의 GC 활성화

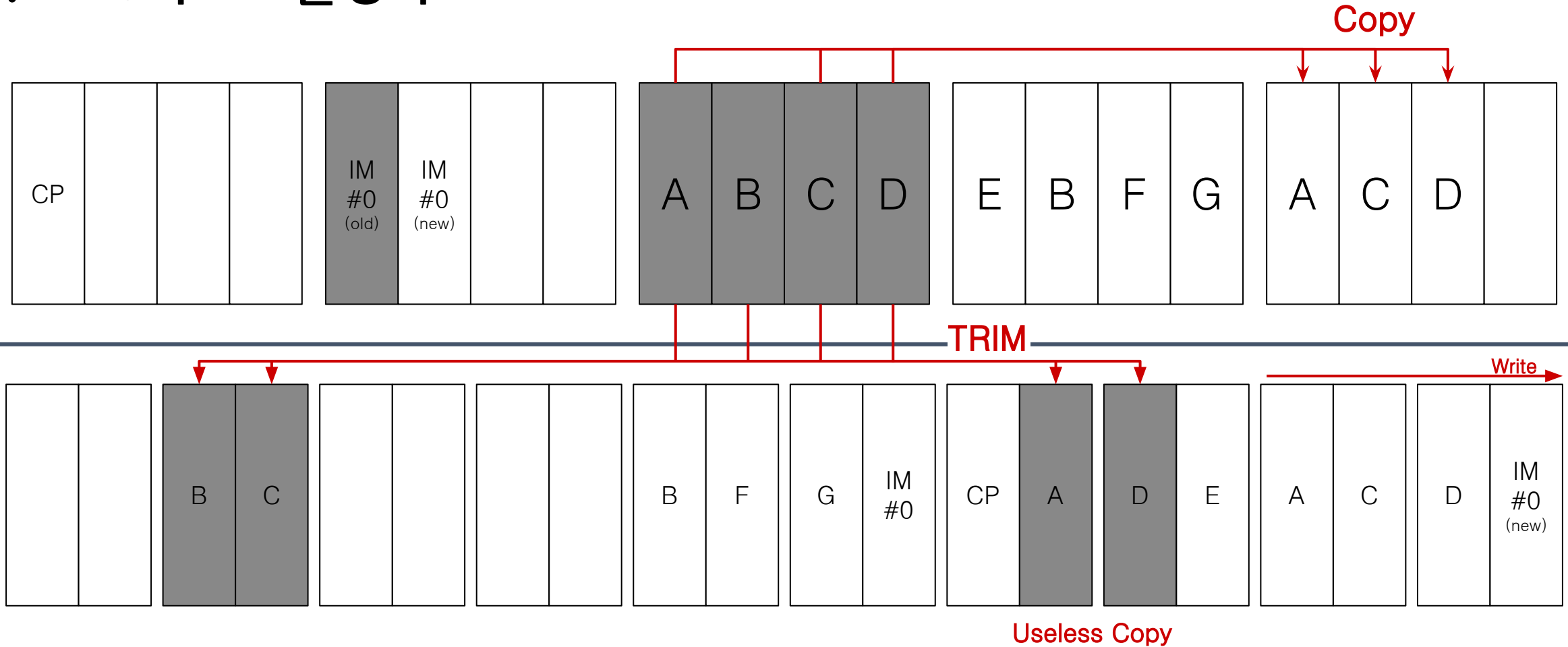


5. Garbage Collection (F2FS + FTL)



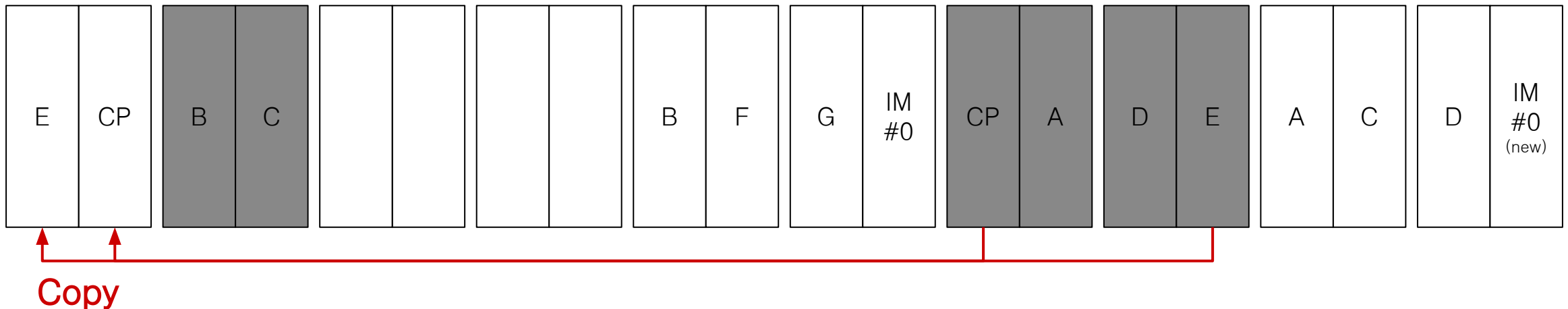
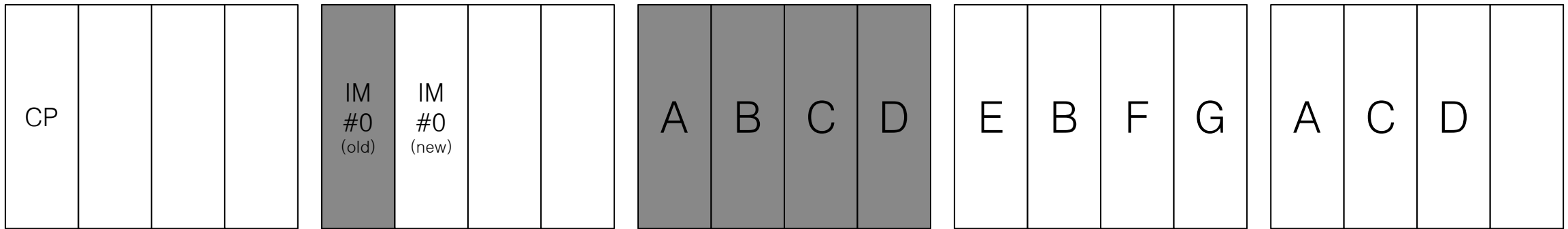
5. Garbage Collection (F2FS + FTL)

2. F2FS의 GC 활성화

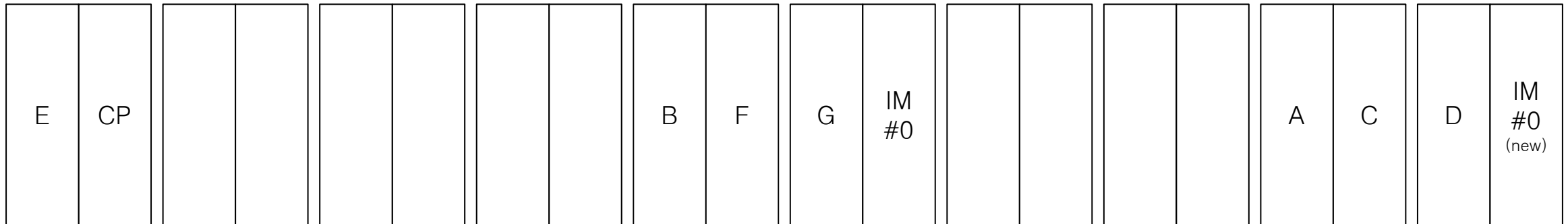
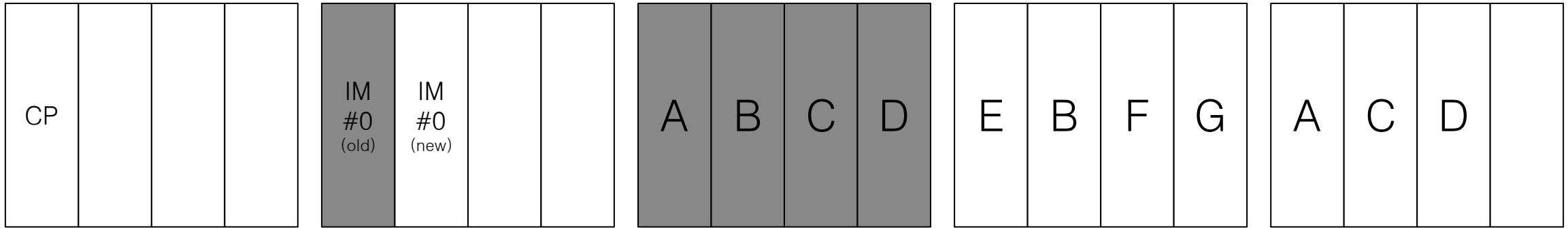


5. Garbage Collection (F2FS + FTL)

3. 다시 FTL의 GC 활성화

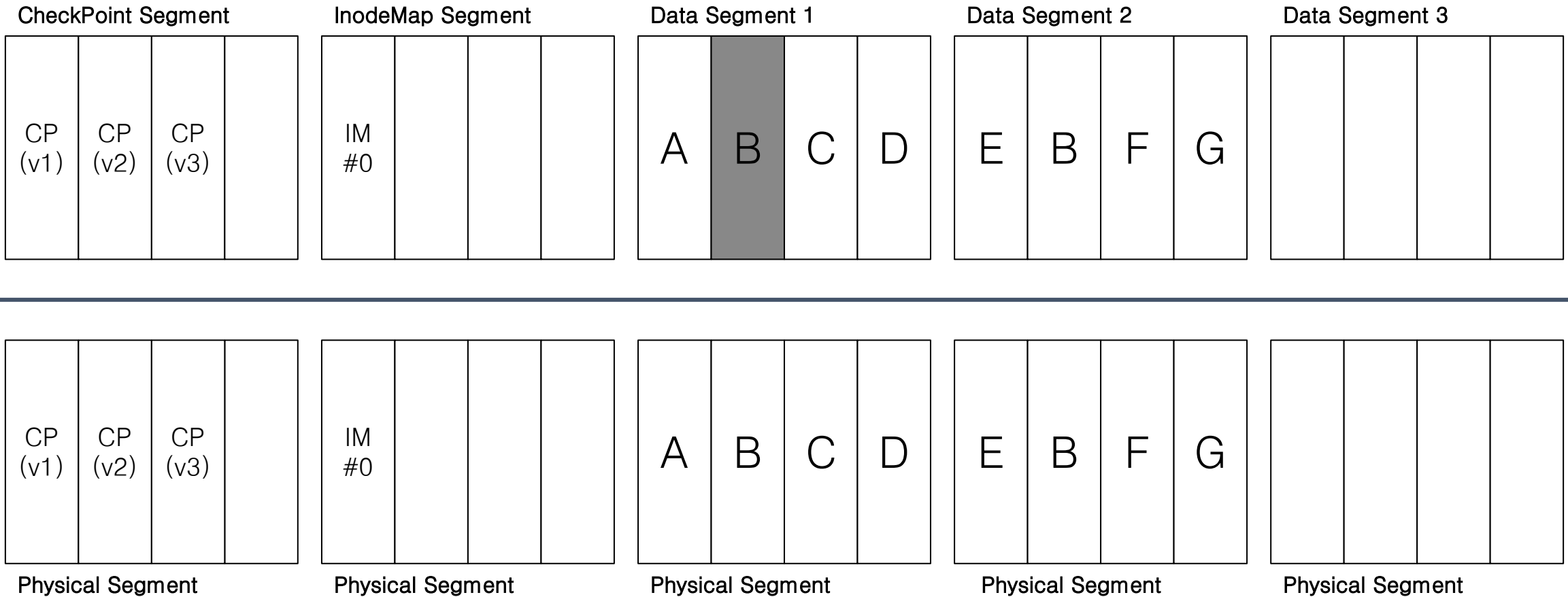


5. Garbage Collection (F2FS + FTL)



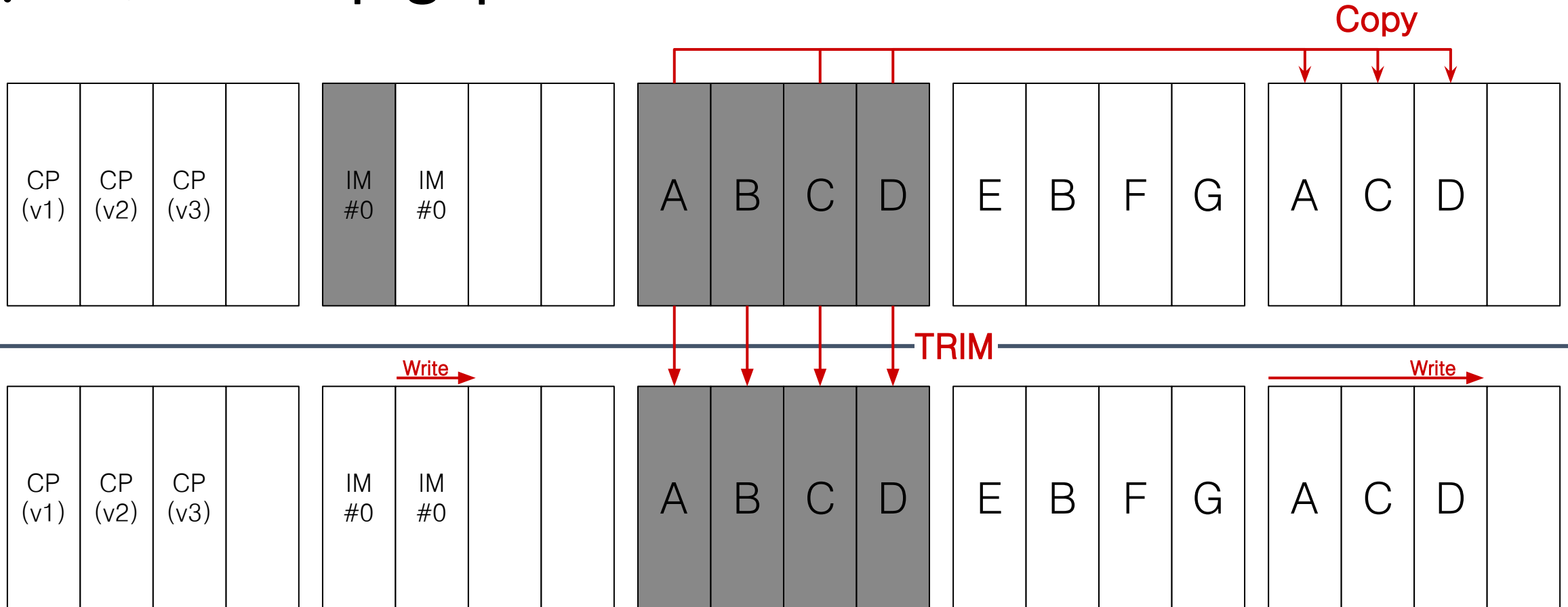
5. Garbage Collection (AMF)

0. 초기 상태

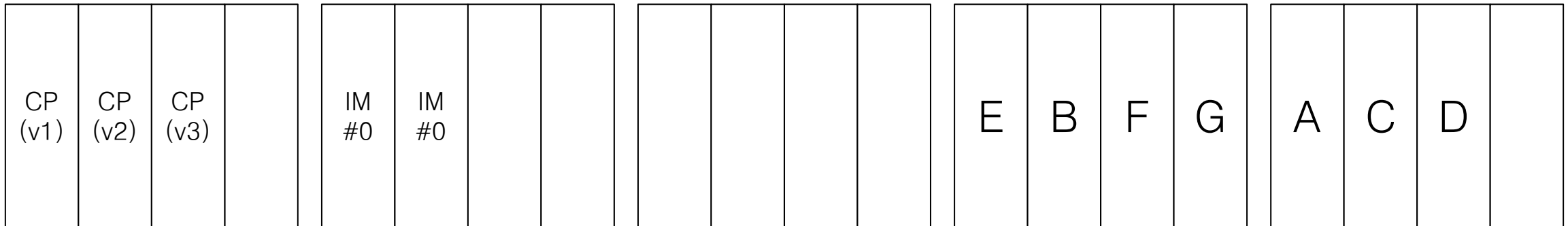
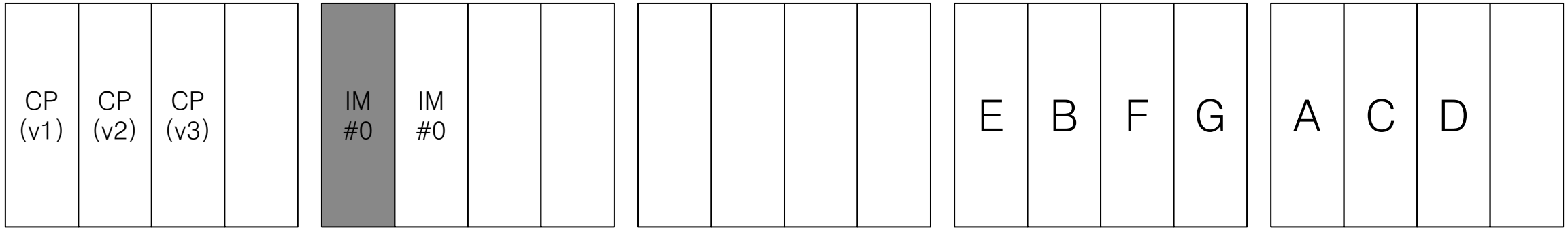


5. Garbage Collection (AMF)

1. ALFS AFTL 모두 정리



5. Garbage Collection (AMF)



6. Experimental Results

- Memory Requirement 비교
 - AFTL / ALFS vs 기존 FTL
- Micro-benchmark 평가
 - 다양한 I/O Access Pattern에서 성능 분석
- Application Benchmark
 - 실제 응용 프로그램에서 성능 평가
- System Analysis
 - SSD 수명, I/O Latency, CPU 사용률

6.1 Memory Requirements

- Mapping Table Size 비교

FTL 방식	512GB	1TB
Block-level FTL	4MB	8MB
Hybrid FTL	96MB	62MB
Page-level FTL	512MB	1GB
AFTL	4MB	8MB

- Hybrid FTL은 전체 용량의 15%를 Over-provisioning 영역으로 사용

6.2 Benchmark Setup

- AMF (ALFS + AFTL),
EXT4 + PFTL, EXT4 + DFTL,
F2FS + PFTL, F2FS + DFTL,

대상 비교

- **PFTL** : Page-level Mapping, 모든 Mapping Entry를 DRAM에 저장
- **DFTL** : 모든 Mapping Entry를 Flash에 저장, Hot Entry 20%만 DRAM에 캐싱, LRU 기반 방식

6.2 Benchmark Setup

- 공통 설정

항목	설정
PFTL / DFTL	Greedy GC
	Over-Provisioning : Storage Capacity의 15%
AFTL Over-Provisioning	불필요
Wear-Leveling	동일한 Dynamic Wear-Leveling
F2FS Segment Size	2MB
ALFS Segment Size	16MB (= Physical Segment Size)
ALFS Inode-map Segment	기존보다 4배 크게 설정
F2FS / ALFS FS Over-Provisioning	5%

6.3 Performance Analysis

- 총 8가지의 Workload에서 각각의 Category 실험 수행

항목	Workload	주요 평가
File System	FIO, Postmark	다양한 File I/O 패턴에서의 성능
DBMS	Non-Trans, OLTP, TPC-C	DB transaction 성능
Hadoop	HDFSIO, TeraSort, WordCount	대규모 batch Processing의 I/O 성능

6.3 Performance Analysis

항목	설정
Host	Intel Xeon, 24 × 1.6 GHz Core, 24 GB DRAM
SSD	8 Channels × 4 Ways
NAND Capacity	512 GB
Page / Block	4 KB × 128 Pages
SSD Raw Performance	Read : 240K IOPS (930 MB/s) / Write : 67K IOPS (260 MB/s)
실험용 Capacity	16 GB
Host DRAM	1.5 GB

GC가 발생하는 aged SSD 환경을 빠르게 모사하기 위해 capacity를 16GB로 설정

6.3.1 Benchmark별 WAF

Garbage Collector가 발생시키는 읽기 증폭 분석

- **F2FS + FTL**
읽기 증폭이 양쪽에서 중복하여 발생
- **EXT4 vs AMF**
일부 Workload에서는 비슷
일부 Workload에서는 AMF가 우위

	EXT4+ PFTL	EXT4+ DFTL	F2FS+ PFTL		F2FS+ DFTL		AMF
	FTL	FTL	FS	FTL	FS	FTL	FS
FIO(SW)	1.00	1.00	1.00	1.00	1.00	1.00	1.00
FIO(RW)	1.41	1.45	1.35	1.82	1.34	2.18	1.38
Postmark(L)	1.00	1.00	1.00	1.00	1.00	1.00	1.00
Postmark(H)	1.12	1.35	1.17	2.23	1.18	2.89	1.16
Non-Trans	1.97	2.00	1.58	2.90	1.59	2.97	1.59
OLTP	1.45	1.46	1.23	1.78	1.23	1.79	1.24
TPC-C	2.33	2.21	1.81	2.80	1.82	5.45	1.87
DFSIO	1.0	1.0	1.0	1.0	1.0	1.0	1.0
TeraSort	1.0	1.0	1.0	1.0	1.0	1.0	1.0
WordCount	1.0	1.0	1.0	1.0	1.0	1.0	1.0

6.3.2 File System Benchmarks

FIO Benchmark

실험 환경

- 10GB 파일 생성 후 다음 4가지 I/O 수행
 - SR (순차 읽기), SW (순차 쓰기), RR (랜덤 읽기), RW (랜덤 쓰기)
- 4KB Block, io-depth : 128
- 8개의 Job 동시 실행 → 순차 / 랜덤 읽기, 쓰기 환경에서의 성능 비교

6.3.2 File System Benchmarks

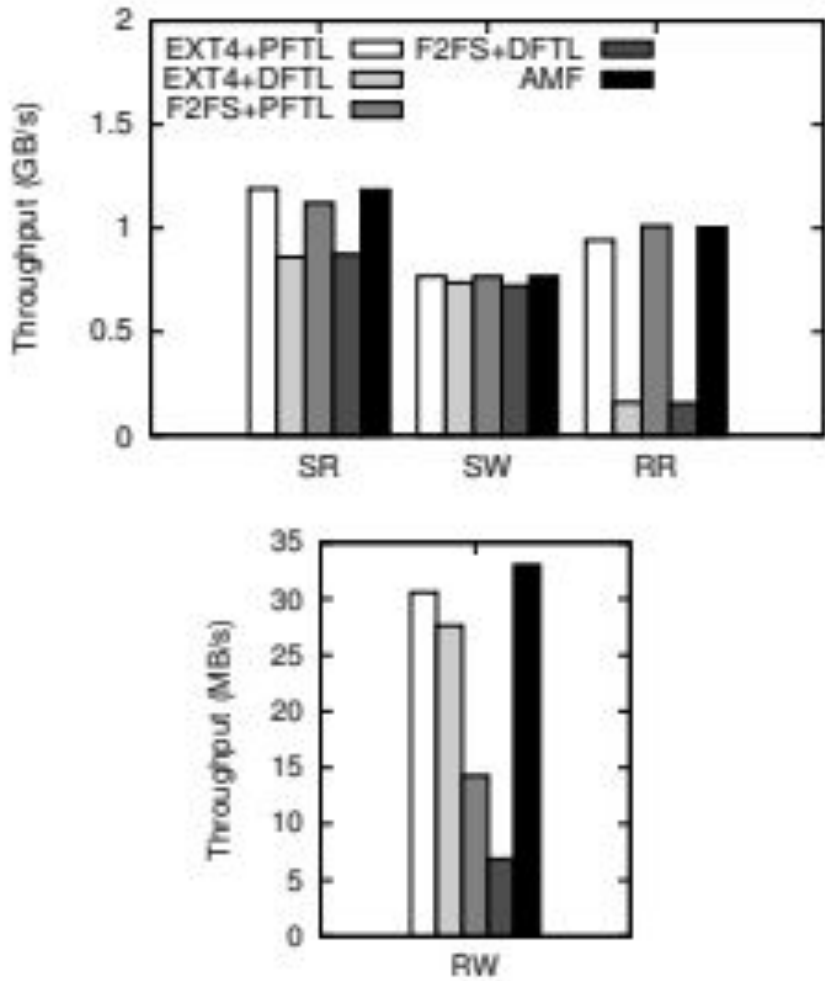


Figure 9: Experimental results with FIO

1. SR, SW

- DFTL 기반은 상대적으로 성능이 떨어지고 이외엔 모두 높은 성능

순차 I/O는 GC로 인해 live Page Copy, Mapping Entry 관리 Overhead가 작음

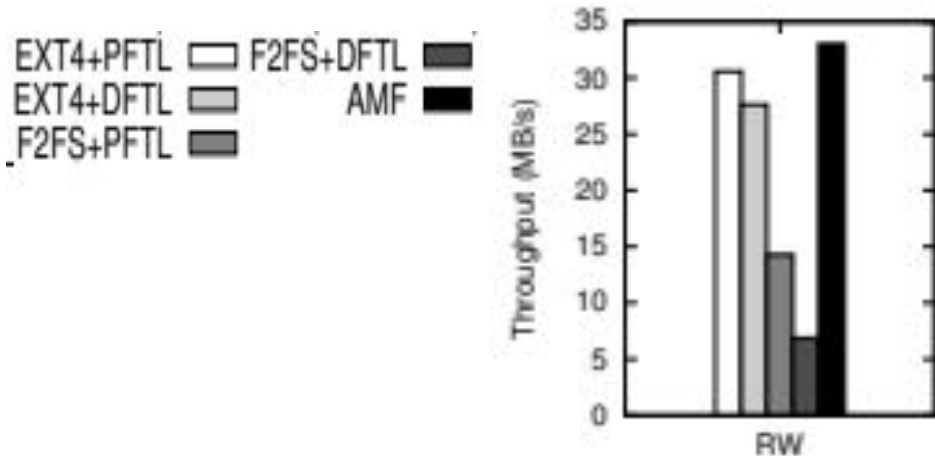
DFTL은 Mapping Entry 일부가 Flash에 있어서 추가적인 I/O가 발생하여 성능이 떨어짐

2. RR

- DFTL 기반에서 크게 낮은 성능

Mapping Entry Cache Miss가 증가하여 Flash에서 Mapping Entry를 읽는 추가 I/O가 발생하기 때문

6.3.2 File System Benchmarks



3. RW

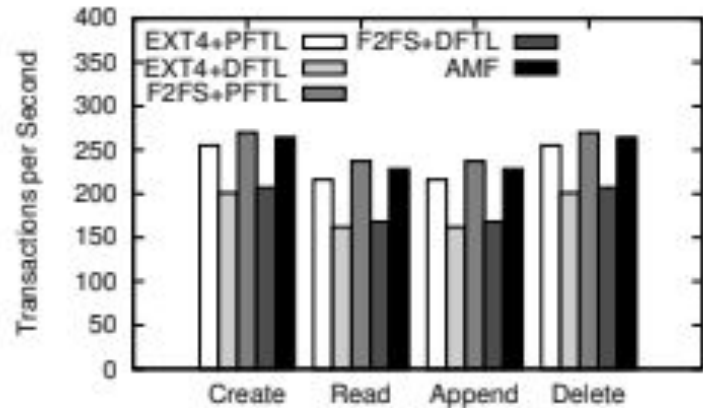
- AMF가 가장 높은 Throughput, 가장 낮은 WAF

AMF는 FTL GC가 따로 없기 때문에 가장 뛰어난 성능

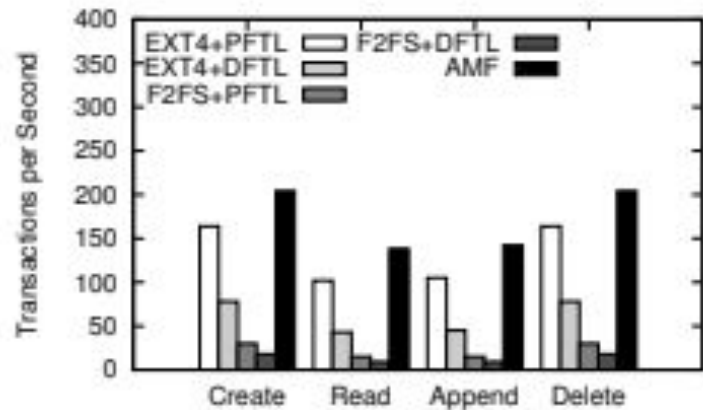
F2FS는 FS에서 Cleaning 후
FTL에서도 다시 GC를 수행하기 때문에
성능이 떨어지는 경향을 보임

	EXT4+ PFTL	EXT4+ DFTL	F2FS+ PFTL		F2FS+ DFTL		AMF
	FTL	FTL	FS	FTL	FS	FTL	FS
FIO(RW)	1.41	1.45	1.35	1.82	1.34	2.18	1.38

6.3.2 File System Benchmarks



(a) Postmark(L)



(b) Postmark(H)

Figure 10: Experimental results with Postmark

(1) Postmark(L) : Light Workload (40%) - GC 적은 환경

- F2FS+PFTL 최고 성능, AMF도 거의 비슷한 성능

GC가 거의 발생하지 않기 때문에 FTL 오버헤드 차이가 거의 없음

(2) Postmark(H) : Heavy Workload (80%) - GC 많은 환경

- AMF가 최고 성능, F2FS+PFTL 성능 크게 감소

Storage가 많이 차있어 GC 자주 발생

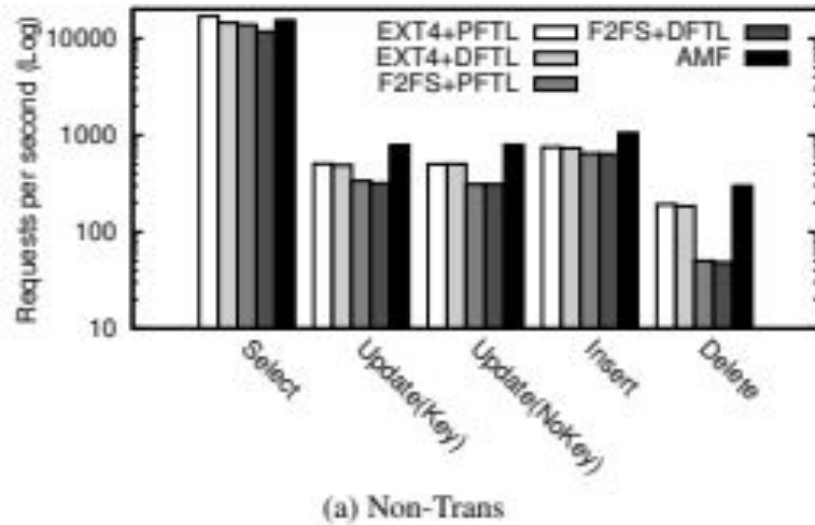
F2FS는 FS와 FTL 모두에서 GC 수행하는 반면,
AMF는 ALFS만 GC 수행

6.3.2 File System Benchmarks

Metadata (Inode Map) 영향

- Postmark는 Metadata 갱신이 매우 많아 Inode Map도 갱신이 빈번
- 하지만 Inode-map Segment 관련 I/O는 전체 I/O의 약 1%기 때문에 Inode Map 관리에 따른 성능 양향은 미미

6.3.3 Application Benchmarks



DataBase Application

각 구성에서 처리한 Transaction 수 비교

- AMF가 가장 높은 Transaction 성능
- AMF는 F2FS의 Cleaning 방식을 활용해 FTL-level GC를 제거
- F2FS + PFTL/DFTL은 GC Overhead로 인하여 낮은 성능을 보임
- Ext4 계열도 AMF보다 낮은 성능

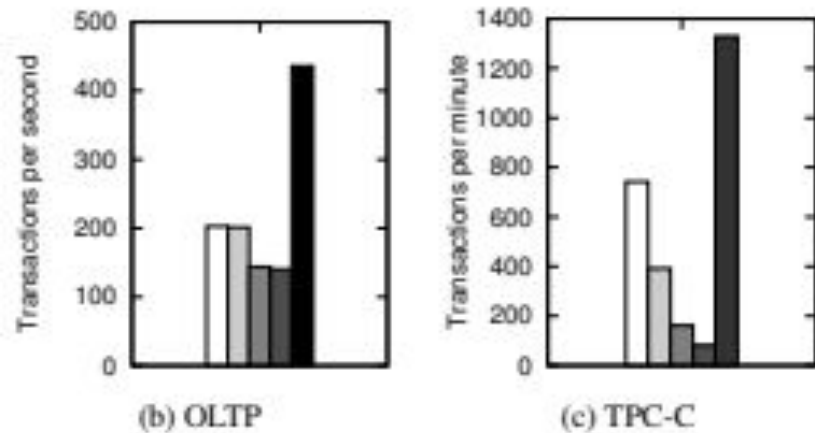
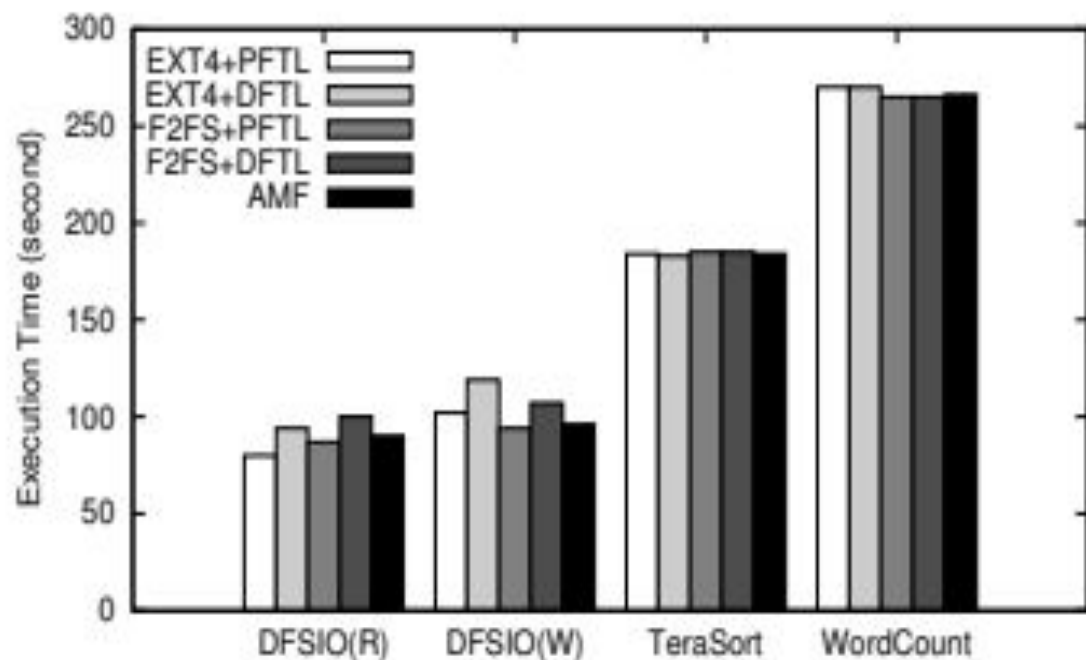


Figure 11: Experimental results with database apps.

6.3.3 Application Benchmarks



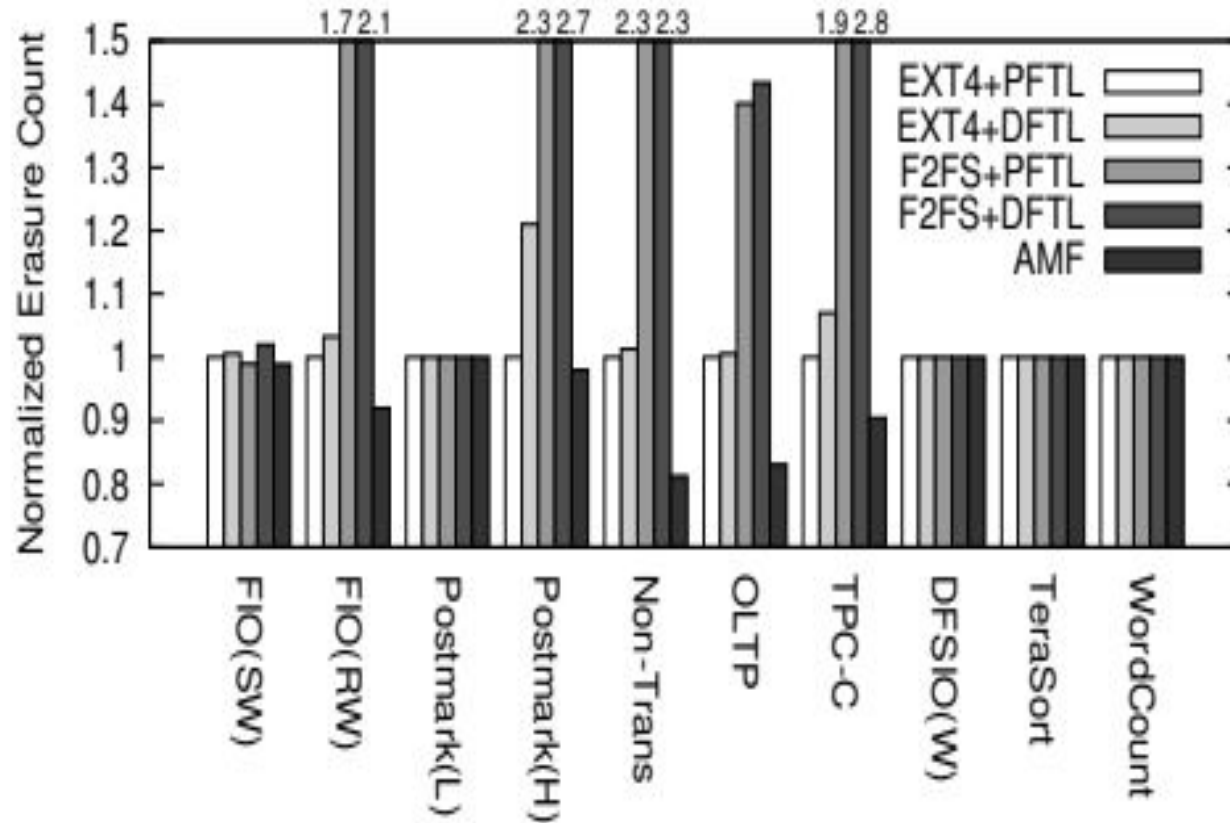
실험 환경 : Hadoop의 HDFS 기반 workload
: HFSIO, TeraSort, WordCount

5가지 구성 모두 비슷한 성능

- HDFS는 대용량 파일을 순차적으로 생성/삭제, 삭제 시 관련 Flash block도 무효화 됨
- FTL GC에서 Live Page Copy가 거의 발생하지 않음
- Mapping Entry miss의 영향도 적음

Figure 12: Experimental results with Hadoop apps.

6.4 Lifetime Analysis



10가지 Write Workload를 대상으로 Flash 수명 분석

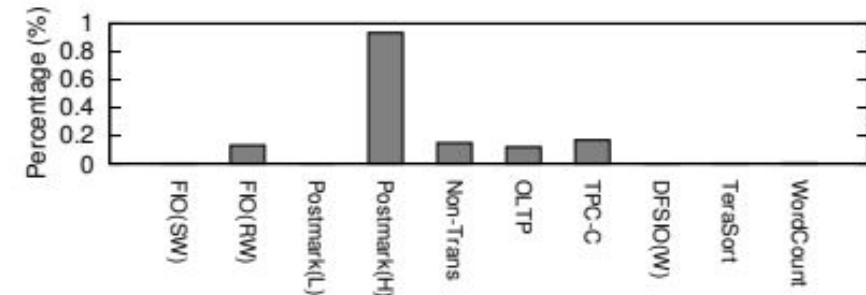
- AMF는 F2FS + DFTL 대비 전체 Erase 횟수 38% 감소

□ Flash의 Erase 부담 감소

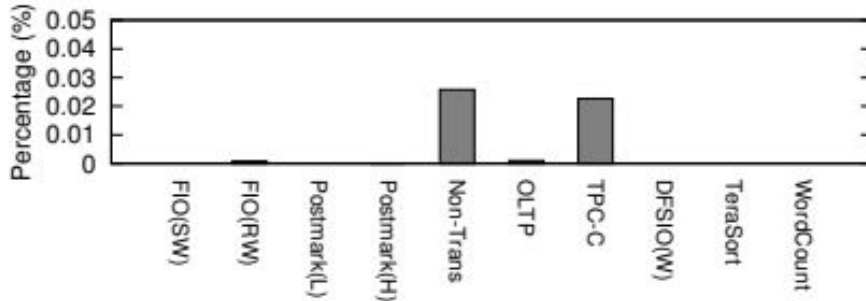
□ Flash Lifetime 향상

Figure 13: Erasure operations normalized to EXT4+PFTL

6.5 Detailed Analysis



(a) TIMB writes



(b) Inode-map garbage collection

- (a) **TIMB Writes** : TIMB에 발생하는 Write가 전체 Write 중 차지하는 비율
- (b) **Inode-map GC** : Inode-map Segment의 GC 과정에서 발생하는 추가 데이터 copy 비율
 - TIMB Write 비율, 추가 Copy 비율 모두 매우 낮음
 - Checkpoint 기록시 평균 2.6개의 dirty TIMB block만 함께 기록됨
- Inode-map을 Append-only 방식으로 관리해서 발생하는 추가 I/O 및 GC 오버헤드는 전체 FS의 오버헤드에 비해 매우 작음

Figure 14: Inode-map management overheads analysis

6.5 Detailed Analysis

CPU : AMF, EXT4, F2FS 모두 유사한 CPU 사용률 (차이 4% 내외)

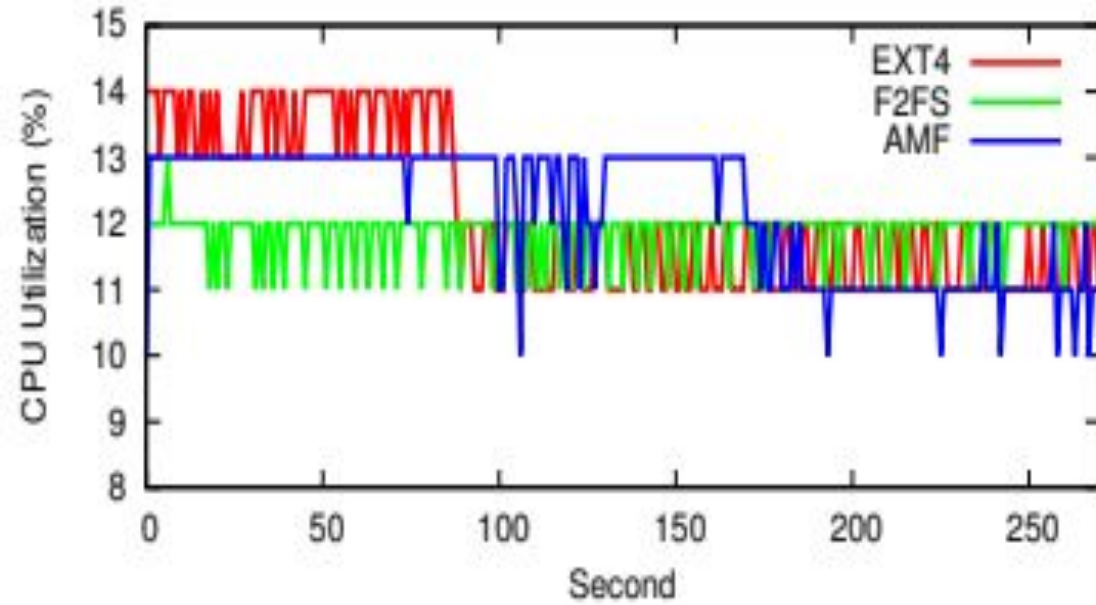


Figure 15: CPU utilization (%)

새로운 layer, 알고리즘 없이 기존 F2FS를 수정하였기 때문에
추가 CPU 오버헤드는 거의 없음

DRAM

16GB SSD F2FS 대비 : + 180KB, 1TB SSD : + 10.8MB
Host Memory 부담도 매우 작음

AMF는 CPU와 DRAM 모두 추가 자원 overhead가 작음

6.5 Detailed Analysis

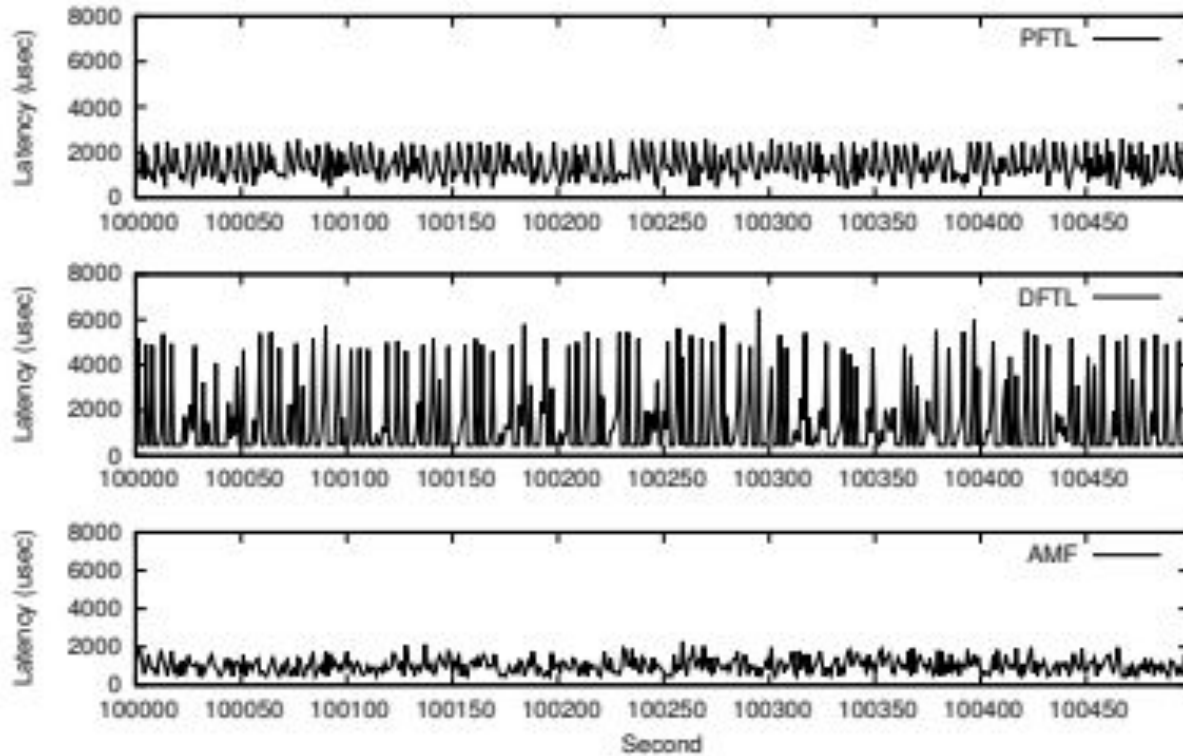


Figure 16: Write latency (μsec)

I/O latency

AMF, PFTL, DFTL의 Response Time을 비교

- AMF가 가장 짧은 응답시간과 작은 Latency 변동

AMF에서는 FTL이 GC, live page copy, Mapping Entry Swap 등 없이 **Block Erase**만 수행

반면 PFTL과 DFTL은 GC, Mapping Entry 관리 때문에 Latency가 크게 변화

7. Related Work

FTL Improvement with Enhanced Interfaces

- I/O에 시스템 수준의 추가 정보를 전달하는 인터페이스
- 장점 : 어떤 데이터가 자주 바뀌는지, 어떤 데이터가 오래 유지되는지 FTL이 알 수 있어 GC를 효율적으로 수행 가능
- 단점 : 추가적인 Hardware 자원 사용, 설계 복잡도 증가
- AMF는 Host(File System)가 Flash를 직접 관리해 추가 Interface나 Device Offloading이 불필요

7. Related Work

Direct Flash Management without FTL

- FFS (Flash File System), NoFTL 등 FTL을 제거하고 Host가 Raw NAND를 직접 관리
- 장점 : FTL이 없으므로 메모리, CPU 사용 감소, 성능 향상
- 단점 : Host가 Bad Block, Wear-Leveling, ECC 등 NAND 특성을 직접 관리해야 함
 - 장치별 구조 차이와 신뢰성 문제로 범용적 적용 어려움
- 논문에서는 Flash File System은 현재 Embedded 환경에서만 주로 사용된다고 설명

7. Related Work

Host-Managed Flash

- Host 기반 FTL : 기존 SSD 내부의 FTL을 Device Driver 계층으로 이동

장점 : FTL을 Host에서 관리 → 저장장치의 Flash 관리에 대한 Host의 제어 및 유연성 향상

단점 : LFS와 Host-based FTL이 각각 GC 수행
→ 기존 FTL 기반 Storage와 동일한 문제 발생

AMF는 FTL을 최소화해 GC와 Address Remapping을 제거, Host의 File System이 Flash를 직접 관리

7. Related Work

SDF (Software Defined Flash)

Flash Channel을 Host에게 직접 노출, 즉 Application이 Flash Channel을 직접 제어

장점 : Datacenter에서 높은 Throughput을 얻을 수 있음

단점 : Custom Interface가 필요해 일반 PC와 SSD에서는 사용하기 어려움

AMF는 READ, WRITE, TRIM만 사용하기 때문에 기존 프로그램도 조금만 수정하면 사용 가능

7. Related Work

REDO

File System과 Flash Device를 통합하여 Flash 관리 효율 및 성능 향상

장점: File System과 Flash를 통합하여 높은 성능 향상을 보임

단점: Metadata 관리, Data Integrity, Multiple Channel 활용 등의 기술적 문제를 충분히 고려하지 않음

AMF와 비교: REDO는 Simulation 기반으로 평가한 반면, AMF는 실제 Linux File System과 SSD Prototype을 구현하여 실험을 수행하여 현실성이 높음

8. Conclusion

- AMF (Application-Managed Flash)라는 아키텍처를 제안
- 신뢰할 수 없는 NAND, Vendor의 세부 사항은 숨기고, Flash Storage를 append-only Segment로 노출하는 Block I/O Interface
- 이를 기반으로 ALFS, AFTL 개발 □ 성능 평가 결과, AMF는 기존 방식보다 적은 자원 사용, 성능 및 수명 증가
- AMF는 DBMS, Key-Value Store (LSM-tree 기반)과 같은 다양한 log-structured system으로 확장 가능

Thank you